CampusPilot AI

Core Source Code

AI-Powered Accessible Campus Navigation | CapGemini Build-a-thon 2026

Repository	https://github.com/AK-1612/campuspilot-ai
Generated	June 14, 2026
License	Apache-2.0

Agent & Intelligence

agent.py

Python | backend/agent/agent.py

```python
import os
import json
from typing import Dict, Any, List, Tuple

from dotenv import load_dotenv
load_dotenv()

from langchain_groq import ChatGroq
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import HumanMessage, SystemMessage, AIMessage, ToolMessage
from .tools import route_query, qr_lookup, profile_detect, flag_obstacle, resolve_room
from .memory import AgentMemory
from .intent_classifier import classify_intent


def load_system_prompt() -> str:
 """
 Loads the system prompt from the markdown file.
 """
 prompt_path = os.path.join(os.path.dirname(__file__), "prompts", "system_prompt.md")
 try:
 with open(prompt_path, "r", encoding="utf-8") as f:
 return f.read()
 except FileNotFoundError:
 return "You are a helpful campus navigation assistant."


def create_campus_agent() -> AgentExecutor:
 """
 Initializes the LangChain tool-calling agent with the Groq LLM
 (llama-3.3-70b-versatile) and the required set of navigation tools.
 """
 llm = ChatGroq(model="llama-3.3-70b-versatile", temperature=0)
 tools = [route_query, qr_lookup, profile_detect, flag_obstacle, resolve_room]
 system_message = load_system_prompt()

 prompt = ChatPromptTemplate.from_messages([
 ("system", system_message),
 ("placeholder", "{chat_history}"),
 ("human", "{input}"),
 ("placeholder", "{agent_scratchpad}"),
 ])

 agent = create_tool_calling_agent(llm, tools, prompt)
 return AgentExecutor(agent=agent, tools=tools, verbose=True, return_intermediate_steps=True)


class CampusPilotAgent:
 """
 The main orchestrator for the CampusPilot AI backend.
 """

 def __init__(self) -> None:
 """Initializes the agent executor and memory layer."""
 self.executor = create_campus_agent()
 self.memory = AgentMemory()

 def process_query(self, query: str, context_overrides: dict = None) -> dict:
 """
 Processes a natural language query from the user.

 Args:
 query (str): The user's input string.
 context_overrides (dict): Optional dict to pass current QR location and profile.

 Returns:
 dict: Contains 'response' (LLM text), 'route_data' (JSON from route tool if any),
 'intent', and 'intermediate_steps_raw' (list of (action, observation) tuples).
 """
 intent = classify_intent(query)
 self.memory.add_interaction("user", query)
 context = self.memory.get_context()
 if context_overrides:
 context.update(context_overrides)

 # Enrich the input with internal state context before passing to the LLM
 enriched_input = (
```

```
 f"[Context: Intent={intent}, "
 f"Profile={context.get('profile', 'default')}, "
 f"Location (QR Code)={context.get('qr_location', 'unknown')}] "
 f"\nUser Query: {query}"
 )

 try:
 result = self.executor.invoke({"input": enriched_input})

 # Extract tool calls from intermediate steps
 intermediate_steps = []
 route_data = None

 for action, observation in result.get("intermediate_steps", []):
 intermediate_steps.append((action, observation))
 if action.tool == "route_query":
 try:
 parsed = json.loads(observation)
 if "route" in parsed:
 route_data = parsed["route"]
 except Exception:
 pass

 response_text = result.get("output", "I could not generate a response.")
 self.memory.add_interaction("assistant", response_text)

 return {
 "response": response_text,
 "route_data": route_data,
 "intent": intent,
 "intermediate_steps_raw": intermediate_steps
 }

 except Exception as e:
 return {
 "response": f"An error occurred while processing your request: {str(e)}",
 "route_data": None,
 "intent": intent,
 "intermediate_steps_raw": []
 }
```

intent_classifier.py Python | backend/agent/intent_classifier.py

```
"""
Pre-LLM intent classifier for the CampusPilot agent.

Performs keyword-based classification in sub-millisecond time before the
LangChain executor is invoked. Emergency intents are detected here so that
safety responses are never delayed by LLM inference latency.
"""

_EMERGENCY_KEYWORDS = frozenset([
 "help", "emergency", "fire", "police", "hurt", "sos", "fell", "stuck",
 "ambulance", "collapse", "attack",
])

_ACCESSIBILITY_KEYWORDS = frozenset([
 "wheelchair", "blind", "deaf", "ramp", "elevator", "lift", "accessible",
 "step-free", "visual impair", "hearing impair",
])

_FACILITY_KEYWORDS = frozenset([
 "where is", "find", "looking for", "locate", "nearest", "closest",
])

_NAVIGATE_KEYWORDS = frozenset([
 "take me", "go to", "route", "navigate", "directions", "how do i get",
 "path to", "way to",
])


def classify_intent(query: str) -> str:
 """
 Classify a user query into one of four system intents.

 Evaluation order: emergency > accessibility_query > find_facility > navigate.
 Emergency takes strict precedence to ensure safety responses are never
 delayed by lower-priority classification branches.

 Args:
 query: Raw text input from the user.

 Returns:
```

```
    One of: 'emergency', 'accessibility_query', 'find_facility', 'navigate'.
    """
    normalised = query.lower()

    if any(kw in normalised for kw in _EMERGENCY_KEYWORDS):
        return "emergency"

    if any(kw in normalised for kw in _ACCESSIBILITY_KEYWORDS):
        return "accessibility_query"

    if any(kw in normalised for kw in _FACILITY_KEYWORDS):
        return "find_facility"

    return "navigate"
```

tools.py Python | backend/agent/tools.py

```
"""
LangChain tool definitions for the CampusPilot agent.

Each tool is registered via the @tool decorator and exposed to the
LangChain AgentExecutor. The LLM autonomously decides which tools to
call and in what order during each ReAct reasoning cycle.
"""

import json
from langchain_core.tools import tool
from backend.db_client import db_client, MOCK_NODES


@tool
def route_query(start_node: str, end_node: str, profile: str) -> str:
    """
    Calculate the shortest accessible route between two campus nodes.

    Args:
    start_node: Exact node ID of the origin (e.g., 'qr-a-0'). Must be
    resolved via qr_lookup before calling this tool.
    end_node: Exact node ID of the destination (e.g., 'room-a-21'). Must be
    resolved via resolve_room before calling this tool.
    profile: Active disability profile (e.g., 'wheelchair', 'vision_impaired').
    Determines which graph edges are eligible.

    Returns:
    JSON string with 'status' and 'route' on success, or 'error' on failure.
    """
    path_data = db_client.find_shortest_path(start_node, end_node, profile)
    if not path_data:
        return json.dumps({
        "error": f"No accessible route found for profile '{profile}' between '{start_node}' and '{end_node}'."
        })
    return json.dumps({"status": "success", "route": path_data})


@tool
def qr_lookup(qr_id: str) -> str:
    """
    Resolve a scanned QR code to its campus node data.

    Args:
    qr_id: The code value embedded in the physical QR marker
    (e.g., 'QR_BUILDING_A_G').

    Returns:
    JSON string with node metadata (id, name, building, floor), or 'error'.
    """
    for node in MOCK_NODES.values():
        if node.get("type") == "QRPoint" and node.get("code") == qr_id:
            return json.dumps(node)

    cypher = (
    "MATCH (q:QRPoint {code: $code}) "
    "RETURN q.id AS id, q.name AS name, q.building AS building, q.floor AS floor"
    )
    try:
        with db_client.get_session() as session:
            result = session.run(cypher, code=qr_id)
            record = result.single()
            if record:
                return json.dumps({
                "id": record["id"],
                "name": record["name"],
                "type": "QRPoint",
```

```
 "building": record["building"],
 "floor": record["floor"],
 })
 except Exception:
 pass

 return json.dumps({"error": f"QR code '{qr_id}' not found."})


@tool
def resolve_room(room_name: str) -> str:
 """
 Look up a room node ID by name or partial description.

 Call this tool before route_query to convert a human-readable destination
 such as 'library' or 'Room 204' into a graph node ID.

 Args:
 room_name: Full or partial name of the room (e.g., 'library', 'AI Lab').

 Returns:
 JSON string with node metadata (id, name, building, floor), or 'error'.
 """
 name_lower = room_name.lower()
 for node in MOCK_NODES.values():
 if node.get("type") == "Room" and name_lower in str(node.get("name", "")).lower():
 return json.dumps(node)

 cypher = (
 "MATCH (r:Room) WHERE toLower(r.name) CONTAINS $name "
 "RETURN r.id AS id, r.name AS name, r.building AS building, r.floor AS floor "
 "LIMIT 1"
 )
 try:
 with db_client.get_session() as session:
 result = session.run(cypher, name=name_lower)
 record = result.single()
 if record:
 return json.dumps({
 "id": record["id"],
 "name": record["name"],
 "type": "Room",
 "building": record["building"],
 "floor": record["floor"],
 })
 except Exception:
 pass

 return json.dumps({"error": f"No room matching '{room_name}' found."})


@tool
def profile_detect(user_input: str) -> str:
 """
 Infer a disability profile from implicit cues in the user's message.

 Used when the user has not explicitly selected a profile but their
 language suggests a specific accessibility need.

 Args:
 user_input: The raw message text from the user.

 Returns:
 A profile name string: 'Wheelchair', 'Vision-impaired',
 'Hearing-impaired', 'Cognitive', or 'Invisible' (default).
 """
 text = user_input.lower()
 if any(kw in text for kw in ("wheelchair", "ramp", "elevator", "step-free")):
 return "Wheelchair"
 if any(kw in text for kw in ("blind", "vision", "visually impaired", "tactile", "audio")):
 return "Vision-impaired"
 if any(kw in text for kw in ("deaf", "hearing", "caption", "visual alert")):
 return "Hearing-impaired"
 if any(kw in text for kw in ("cognitive", "simple", "slow", "one step")):
 return "Cognitive"
 return "Invisible"


@tool
def flag_obstacle(node_id: str, description: str) -> str:
 """
 Report a temporary obstacle and mark the affected node on the shadow map.

```

```
The shadow map layer blocks flagged nodes from appearing in future route
calculations until the obstacle is cleared.

Args:
node_id: Exact graph node ID of the blocked location
(e.g., 'lift-a', 'corr-a-0').
description: Plain-language description of the obstacle.

Returns:
Confirmation string.
"""
db_client.flag_shadow_node(node_id)
return (
f"Obstacle recorded at node '{node_id}': {description}. "
"The node has been flagged on the shadow map and will be excluded from routing."
)
```

memory.py Python | backend/agent/memory.py

```python
from typing import List, Dict, Any, Optional

class AgentMemory:
 """
 A lightweight state management layer for the CampusPilot agent.

 Tracks conversational history, the active disability profile,
 and the last known physical location based on QR scans.
 """

 def __init__(self) -> None:
 """Initializes empty memory structures and default settings."""
 self.history: List[Dict[str, str]] = []
 self.last_known_qr_location: Optional[str] = None
 self.current_profile: str = "Invisible"

 def add_interaction(self, role: str, content: str) -> None:
 """
 Records a message to the conversational history.

 Args:
 role (str): The role of the speaker (e.g., 'user', 'assistant').
 content (str): The message text.
 """
 self.history.append({"role": role, "content": content})

 def update_location(self, qr_id: str, location_name: str) -> None:
 """
 Updates the user's last known physical location.

 Args:
 qr_id (str): The ID of the scanned QR code.
 location_name (str): The human-readable name of the location.
 """
 self.last_known_qr_location = location_name

 def update_profile(self, profile: str) -> None:
 """
 Updates the active disability routing profile for the session.

 Args:
 profile (str): The validated profile name.
 """
 self.current_profile = profile

 def get_context(self) -> Dict[str, Any]:
 """
 Retrieves the current session context summarizing the user's state.

 Returns:
 Dict[str, Any]: A dictionary containing location, profile, and history metadata.
 """
 return {
 "last_location": self.last_known_qr_location,
 "profile": self.current_profile,
 "history_length": len(self.history)
 }
```

profile_handler.py Python | backend/agent/profile_handler.py

```python
from enum import Enum
from typing import Dict

class DisabilityProfile(Enum):
```

```
    """Enumeration of the supported user disability profiles."""
    WHEELCHAIR = "Wheelchair"
    VISION_IMPAIRED = "Vision-impaired"
    HEARING_IMPAIRED = "Hearing-impaired"
    COGNITIVE = "Cognitive"
    INVISIBLE = "Invisible"


def get_routing_constraints(profile: DisabilityProfile) -> Dict[str, bool]:
    """
    Maps a disability profile to specific parameters for the routing engine.

    Args:
    profile (DisabilityProfile): The active profile enum.

    Returns:
    Dict[str, bool]: A dictionary of boolean constraint flags to be passed
    to the Neo4j pathfinding algorithm.
    """
    constraints: Dict[str, bool] = {
    "avoid_stairs": False,
    "prefer_elevators": False,
    "detailed_landmarks": False,
    "visual_cues_only": False,
    "simplified_steps": False
    }

    if profile == DisabilityProfile.WHEELCHAIR:
    constraints["avoid_stairs"] = True
    constraints["prefer_elevators"] = True
    elif profile == DisabilityProfile.VISION_IMPAIRED:
    constraints["detailed_landmarks"] = True
    elif profile == DisabilityProfile.HEARING_IMPAIRED:
    constraints["visual_cues_only"] = True
    elif profile == DisabilityProfile.COGNITIVE:
    constraints["simplified_steps"] = True

    return constraints


def parse_profile(profile_str: str) -> DisabilityProfile:
    """
    Safely parses a string into a DisabilityProfile enum.

    Args:
    profile_str (str): The raw profile string (e.g., from an API or database).

    Returns:
    DisabilityProfile: The matched enum, or DisabilityProfile.INVISIBLE as a safe fallback.
    """
    try:
    return DisabilityProfile(profile_str)
    except ValueError:
    return DisabilityProfile.INVISIBLE
```

fallbacks.py Python | backend/agent/fallbacks.py

```
"""
Fallback response generators for the CampusPilot agent.

These functions provide safe, pre-approved responses for situations where
the routing engine cannot produce a result or where the LLM must be bypassed
entirely (e.g., emergency detection).
"""


def handle_emergency() -> str:
    """
    Returns the standard emergency response.

    This response is returned immediately when an emergency intent is detected
    by the pre-LLM heuristic classifier. It does not depend on LLM inference.
    """
    return (
    "Emergency detected. Campus Security has been alerted. "
    "Please stay calm and proceed to the nearest marked exit. "
    "If you require immediate assistance, call campus security at extension 100."
    )


def handle_ambiguous_query(query: str) -> str:
    """
    Returns a clarification request when the agent cannot determine intent.
```

```
    Args:
    query: The ambiguous user query.

    Returns:
    A plain-language request for more specific information.
    """
    return (
    "I was unable to determine a destination from your request. "
    "Please specify a building name, room number, or facility type."
    )


def handle_no_route_found(start: str, end: str) -> str:
    """
    Returns a safe response when no accessible route exists between two nodes.

    Args:
    start: Starting node ID or label.
    end: Destination node ID or label.

    Returns:
    A fallback message with guidance for the user.
    """
    return (
    f"No accessible route could be found from {start} to {end} at this time. "
    "This may be due to a temporary closure or missing accessibility data. "
    "Please contact campus facilities or speak to a member of staff."
    )


def handle_offline_mode() -> str:
    """
    Returns the standard notification for offline or degraded-connectivity operation.

    Returns:
    An informational message about reduced functionality.
    """
    return (
    "The navigation service is currently operating offline. "
    "Cached routes from your last QR scan are available, "
    "but real-time updates require network connectivity."
    )
```

API & Routing

main.py

Python | backend/main.py

```python
"""
CampusPilot AI — FastAPI Application Entry Point

Configures the FastAPI application, CORS middleware, and API routers.
"""

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from backend.routers import qr, navigate, alert

app = FastAPI(
 title="CampusPilot AI API",
 description=(
 "Backend services for accessible indoor campus navigation. "
 "Provides graph-based routing, QR checkpoint resolution, "
 "and agentic wayfinding via LangChain and Neo4j."
 ),
 version="1.0.0",
 contact={
 "name": "CampusPilot AI Team",
 "url": "https://github.com/AK-1612/campuspilot-ai",
 },
 license_info={"name": "Apache-2.0"},
)

# CORS — restrict in production to your frontend's origin
app.add_middleware(
 CORSMiddleware,
 allow_origins=["*"],
 allow_credentials=True,
 allow_methods=["*"],
 allow_headers=["*"],
)

app.include_router(qr.router)
app.include_router(navigate.router)
app.include_router(alert.router)


@app.get("/", tags=["Health"])
def health_check() -> dict:
 """Returns API health status."""
 return {"status": "healthy", "service": "CampusPilot AI API", "version": "1.0.0"}


if __name__ == "__main__":
 import uvicorn
 uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

navigate.py

Python | backend/routers/navigate.py

```python
"""
Navigation router — POST /navigate

Accepts a natural-language query and routes it through the CampusPilot agent.
Emergency intents are short-circuited before the LLM is invoked.
"""

from fastapi import APIRouter
from pydantic import BaseModel
from typing import List, Optional, Any

from backend.agent.agent import CampusPilotAgent
from backend.agent.intent_classifier import classify_intent
from backend.agent.fallbacks import handle_emergency

router = APIRouter(prefix="/navigate", tags=["Navigation"])

# Agent is instantiated once at startup; AgentExecutor is thread-safe for reads.
_agent = CampusPilotAgent()

# Maximum output length for tool observations in API responses
MAX_OBSERVATION_LENGTH = 400


class AgentStep(BaseModel):
 """A single tool invocation recorded during an agent reasoning cycle."""
 tool: str
```

```
 input: str
 output: str


class NavigateRequest(BaseModel):
 origin: str
 destination: str
 profile: str
 qr_location: Optional[str] = None


class NavigateResponse(BaseModel):
 response: str
 route_data: Optional[Any] = None
 agent_steps: List[AgentStep] = []
 intent: Optional[str] = None
 profile_applied: Optional[str] = None


@router.post("", response_model=NavigateResponse)
@router.post("/", response_model=NavigateResponse)
def navigate(req: NavigateRequest) -> NavigateResponse:
 """
 Process a navigation query through the CampusPilot agentic pipeline.

 Emergency intents are detected by a sub-millisecond heuristic classifier
 before the LLM is invoked, ensuring immediate safety responses.
 All other intents are resolved by the LangChain ReAct agent using the
 registered tool set (qr_lookup, resolve_room, route_query, flag_obstacle).
 """
 intent = classify_intent(req.destination)

 if intent == "emergency":
 return NavigateResponse(
 response=handle_emergency(),
 agent_steps=[AgentStep(
 tool="emergency_classifier",
 input=req.destination,
 output="Emergency intent detected via pre-LLM heuristic. LLM invocation bypassed."
 )],
 intent="emergency",
 profile_applied=req.profile,
 )

 context_overrides = {
 "qr_location": req.qr_location or req.origin,
 "profile": req.profile,
 }

 result = _agent.process_query(req.destination, context_overrides)

 steps: List[AgentStep] = []
 for action, observation in result.get("intermediate_steps_raw", []):
 steps.append(AgentStep(
 tool=action.tool,
 input=str(action.tool_input),
 output=str(observation)[:MAX_OBSERVATION_LENGTH],
 ))

 return NavigateResponse(
 response=result.get("response", ""),
 route_data=result.get("route_data"),
 agent_steps=steps,
 intent=result.get("intent"),
 profile_applied=req.profile,
 )
```

qr.py Python | backend/routers/qr.py

```
"""
QR router — GET /qr/lookup

Resolves a scanned QR code to its campus node metadata.
"""

import logging
from fastapi import APIRouter, HTTPException, Query
from pydantic import BaseModel
from typing import Optional

from backend.db_client import db_client, MOCK_NODES

logger = logging.getLogger(__name__)
```

```python
router = APIRouter(prefix="/qr", tags=["QR Codes"])


class QRLocationResponse(BaseModel):
 id: str
 code: str
 name: str
 building: str
 floor: int
 description: Optional[str] = None


@router.get("/lookup", response_model=QRLocationResponse)
def lookup_qr(
 code: str = Query(..., description="QR code value scanned from the physical marker.")
) -> QRLocationResponse:
 """
 Resolve a QR code to its associated campus location node.

 Attempts resolution against Neo4j first, then falls back to the
 in-memory mock graph if the database is unavailable.
 """
 cypher = """
 MATCH (q:QRPoint {code: $code})
 RETURN q.id AS id, q.code AS code, q.name AS name,
 q.building AS building, q.floor AS floor,
 q.description AS description
 """
 try:
 with db_client.get_session() as session:
 result = session.run(cypher, code=code)
 record = result.single()
 if record:
 return QRLocationResponse(
 id=record["id"],
 code=record["code"],
 name=record["name"],
 building=record["building"],
 floor=record["floor"],
 description=record["description"],
 )
 except Exception as exc:
 logger.warning("Neo4j QR lookup failed, using mock fallback: %s", exc)

 for node in MOCK_NODES.values():
 if node.get("type") == "QRPoint" and node.get("code") == code:
 return QRLocationResponse(
 id=node["id"],
 code=node["code"],
 name=node["name"],
 building=node["building"],
 floor=node["floor"],
 description=node.get("description"),
 )

 raise HTTPException(
 status_code=404,
 detail=f"QR code '{code}' was not recognised. Verify the code and try again.",
 )
```

alert.py Python | backend/routers/alert.py

```python
"""
Alert router — POST /alert

Receives hazard and obstacle reports from the frontend.
In production this would persist to a database and trigger re-routing.
"""

import logging
from fastapi import APIRouter
from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime, timezone

logger = logging.getLogger(__name__)
router = APIRouter(prefix="/alert", tags=["Alerts"])


class AlertRequest(BaseModel):
 user: str = Field(..., description="Identifier of the reporting user or session.")
 user_id: Optional[str] = Field(None, alias="userId")
 description: str = Field(..., description="Description of the hazard or obstacle.")
```

```
 location: Optional[str] = Field(None, description="Human-readable location string.")
 is_custom: bool = Field(True, alias="isCustom")

 class Config:
 populate_by_name = True


class AlertResponse(BaseModel):
 status: str
 alert_id: str
 timestamp: str
 message: str


@router.post("", response_model=AlertResponse)
def report_alert(alert: AlertRequest) -> AlertResponse:
 """
 Record a campus hazard or obstacle report.

 The report is acknowledged and logged. In a production deployment this
 endpoint would persist the incident and propagate it to the shadow map
 routing layer so affected paths are recalculated in real time.
 """
 import uuid
 alert_id = f"ALT-{uuid.uuid4().hex[:8].upper()}"
 timestamp = datetime.now(timezone.utc).isoformat()

 logger.info(
 "Alert received | id=%s | location=%s | description=%s",
 alert_id,
 alert.location,
 alert.description,
 )

 return AlertResponse(
 status="received",
 alert_id=alert_id,
 timestamp=timestamp,
 message="Hazard report recorded. Routing engine has been notified.",
 )
```

config.py Python | backend/config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

class Settings:
 NEO4J_URI = os.getenv("NEO4J_URI", "bolt://localhost:7687")
 NEO4J_USER = os.getenv("NEO4J_USER", "neo4j")
 NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD")
 GROQ_API_KEY = os.getenv("GROQ_API_KEY", "")

settings = Settings()
```

Database

schema.cypher

Cypher | db/schema.cypher

```cypher
// CampusPilot AI — Neo4j Schema Definitions
// Version: 1.0

// Constraints
CREATE CONSTRAINT UNIQUE_LOCATION_ID IF NOT EXISTS
FOR (l:Location) REQUIRE l.id IS UNIQUE;

CREATE CONSTRAINT UNIQUE_QR_ID IF NOT EXISTS
FOR (q:QRPoint) REQUIRE q.id IS UNIQUE;

// Indexes
CREATE INDEX LOCATION_NAME_INDEX IF NOT EXISTS
FOR (l:Location) ON (l.name);

CREATE INDEX QR_CODE_INDEX IF NOT EXISTS
FOR (q:QRPoint) ON (q.code);

// Nodes:
// - QRPoint {id, code, name, building, floor, description}
// - Room {id, name, building, floor, type}
// - Corridor {id, building, floor, name}
// - Lift {id, building, name}
// - Stairs {id, building, name}
// - Entrance {id, building, name}

// Relationships:
// - (:QRPoint)-[:CONNECTS_TO {distance_meters: Float, wheelchair_accessible: Boolean, tactile_paving: Boolean, stairs: Boolean, lift: Boolean}]-…
```

seed.cypher

Cypher | db/seed.cypher

```cypher
// Seed Script for CampusPilot AI Graph Database
// Contains 2 buildings (Building A and Building B)
// 3 floors per building, 10+ rooms, lifts, stairs, entrances, and QR nodes.

// Clear existing database contents
MATCH (n) DETACH DELETE n;

// --- BUILDING A ---
// Ground Floor (Floor 0)
CREATE (entA:Entrance {id: "ent-a-0", building: "Building A", name: "Main Entrance A"})
CREATE (qrA0:QRPoint {id: "qr-a-0", code: "QR_BUILDING_A_G", name: "Building A Ground Floor Lobby QR", building: "Building A",
CREATE (qrA0:QRPoint {id: "qr-a-0", code: "QR_BUILDING_A_G", name: "Building A Ground Floor Lobby QR", building: "Building A",
CREATE (corrA0:Corridor {id: "corr-a-0", building: "Building A", floor: 0, name: "Ground Floor Corridor"})
CREATE (roomA01:Room {id: "room-a-01", name: "Seminar Hall 101", building: "Building A", floor: 0, type: "Seminar Hall"})
CREATE (roomA02:Room {id: "room-a-02", name: "Admission Office 102", building: "Building A", floor: 0, type: "Office"})
CREATE (liftA:Lift {id: "lift-a", building: "Building A", name: "Main Elevator A"})
CREATE (stairsA:Stairs {id: "stairs-a", building: "Building A", name: "Staircase A"})

// First Floor (Floor 1)
CREATE (qrA1:QRPoint {id: "qr-a-1", code: "QR_BUILDING_A_1", name: "Building A First Floor QR", building: "Building A", floor:
CREATE (qrA1:QRPoint {id: "qr-a-1", code: "QR_BUILDING_A_1", name: "Building A First Floor QR", building: "Building A", floor:
CREATE (corrA1:Corridor {id: "corr-a-1", building: "Building A", floor: 1, name: "First Floor Corridor"})
CREATE (roomA11:Room {id: "room-a-11", name: "AI Lab 201", building: "Building A", floor: 1, type: "Laboratory"})
CREATE (roomA12:Room {id: "room-a-12", name: "Classroom 202", building: "Building A", floor: 1, type: "Classroom"})

// Second Floor (Floor 2)
CREATE (qrA2:QRPoint {id: "qr-a-2", code: "QR_BUILDING_A_2", name: "Building A Second Floor QR", building: "Building A",
CREATE (qrA2:QRPoint {id: "qr-a-2", code: "QR_BUILDING_A_2", name: "Building A Second Floor QR", building: "Building A",
CREATE (corrA2:Corridor {id: "corr-a-2", building: "Building A", floor: 2, name: "Second Floor Corridor"})
CREATE (roomA21:Room {id: "room-a-21", name: "Library", building: "Building A", floor: 2, type: "Library"})
CREATE (roomA22:Room {id: "room-a-22", name: "Server Room 302", building: "Building A", floor: 2, type: "Server Room"})

// --- BUILDING B ---
// Ground Floor (Floor 0)
CREATE (entB:Entrance {id: "ent-b-0", building: "Building B", name: "Main Entrance B"})
CREATE (qrB0:QRPoint {id: "qr-b-0", code: "QR_BUILDING_B_G", name: "Building B Ground Floor Lobby QR", building: "Building B",
CREATE (qrB0:QRPoint {id: "qr-b-0", code: "QR_BUILDING_B_G", name: "Building B Ground Floor Lobby QR", building: "Building B",
CREATE (corrB0:Corridor {id: "corr-b-0", building: "Building B", floor: 0, name: "Ground Floor Corridor B"})
CREATE (roomB01:Room {id: "room-b-01", name: "Auditorium 1", building: "Building B", floor: 0, type: "Auditorium"})
CREATE (roomB02:Room {id: "room-b-02", name: "Cafeteria", building: "Building B", floor: 0, type: "Food Court"})
CREATE (liftB:Lift {id: "lift-b", building: "Building B", name: "Elevator B"})
CREATE (stairsB:Stairs {id: "stairs-b", building: "Building B", name: "Staircase B"})

// First Floor (Floor 1)
CREATE (qrB1:QRPoint {id: "qr-b-1", code: "QR_BUILDING_B_1", name: "Building B First Floor QR", building: "Building B", floor:
CREATE (qrB1:QRPoint {id: "qr-b-1", code: "QR_BUILDING_B_1", name: "Building B First Floor QR", building: "Building B", floor:
CREATE (corrB1:Corridor {id: "corr-b-1", building: "Building B", floor: 1, name: "First Floor Corridor B"})
CREATE (roomB11:Room {id: "room-b-11", name: "Physics Lab 201", building: "Building B", floor: 1, type: "Laboratory"})
CREATE (roomB12:Room {id: "room-b-12", name: "Classroom 203", building: "Building B", floor: 1, type: "Classroom"})

// Second Floor (Floor 2)
CREATE (qrB2:QRPoint {id: "qr-b-2", code: "QR_BUILDING_B_2", name: "Building B Second Floor QR", building: "Building B",
CREATE (qrB2:QRPoint {id: "qr-b-2", code: "QR_BUILDING_B_2", name: "Building B Second Floor QR", building: "Building B",
CREATE (corrB2:Corridor {id: "corr-b-2", building: "Building B", floor: 2, name: "Second Floor Corridor B"})
```

```
CREATE (roomB21:Room {id: "room-b-21", name: "Dean's Office", building: "Building B", floor: 2, type: "Office"})
CREATE (roomB22:Room {id: "room-b-22", name: "Conference Room 305", building: "Building B", floor: 2, type: "Meeting Room"})

// --- CONNECTIONS WITHIN BUILDING A ---
// Ground Floor Connections
CREATE (entA)-[:CONNECTS_TO {distance_meters: 5.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (qrA0)-[:CONNECTS_TO {distance_meters: 5.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
false}]->(entA)
CREATE (qrA0)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (corA0)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
false}]->(qrA0)
CREATE (corrA0)-[:CONNECTS_TO {distance_meters: 15.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
CREATE (roomA01)-[:CONNECTS_TO {distance_meters: 15.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
lift: false}]->(corrA0)
CREATE (corrA0)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
CREATE (roomA02)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
false}]->(corrA0)
// Vertical Connections (Lift A) - Accessible
CREATE (qrA0)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (liftA)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
true}]->(qrA0)
// Vertical Connections (Stairs A) - Non-Accessible
CREATE (corrA0)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
CREATE (stairsA)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
false}]->(corrA0)
// First Floor Connections
CREATE (qrA1)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (corA1)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
false}]->(qrA1)
CREATE (corrA1)-[:CONNECTS_TO {distance_meters: 12.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
CREATE (roomA11)-[:CONNECTS_TO {distance_meters: 12.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
lift: false}]->(corrA1)
CREATE (corrA1)-[:CONNECTS_TO {distance_meters: 9.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
CREATE (roomA12)-[:CONNECTS_TO {distance_meters: 9.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
false}]->(corrA1)
CREATE (qrA1)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (liftA)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
true}]->(qrA1)
CREATE (corrA1)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
CREATE (stairsA)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
false}]->(corrA1)
// Second Floor Connections
CREATE (qrA2)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (corA2)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
false}]->(qrA2)
CREATE (corrA2)-[:CONNECTS_TO {distance_meters: 14.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (roomA21)-[:CONNECTS_TO {distance_meters: 14.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
false}]->(corrA2)
CREATE (corrA2)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
CREATE (roomA22)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
false}]->(corrA2)
CREATE (qrA2)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (liftA)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
true}]->(qrA2)
CREATE (corrA2)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
CREATE (stairsA)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
false}]->(corrA2)
// --- CONNECTIONS WITHIN BUILDING B ---
// Ground Floor Connections
CREATE (entB)-[:CONNECTS_TO {distance_meters: 5.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (qrB0)-[:CONNECTS_TO {distance_meters: 5.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
false}]->(entB)
CREATE (qrB0)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (corB0)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
false}]->(qrB0)
CREATE (corrB0)-[:CONNECTS_TO {distance_meters: 15.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
CREATE (roomB01)-[:CONNECTS_TO {distance_meters: 15.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
lift: false}]->(corrB0)
CREATE (corrB0)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
CREATE (roomB02)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
false}]->(corrB0)
CREATE (qrB0)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (liftB)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
true}]->(qrB0)
CREATE (corrB0)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
CREATE (stairsB)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
false}]->(corrB0)
// First Floor Connections
CREATE (qrB1)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
CREATE (corB1)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
false}]->(qrB1)
CREATE (corrB1)-[:CONNECTS_TO {distance_meters: 12.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
CREATE (roomB11)-[:CONNECTS_TO {distance_meters: 12.0, wheelchair_accessible: true, tactile_paving: false, stairs: false,
lift: false}]->(corrB1)
```

```
132
133 CREATE (corrB1)-[:CONNECTS_TO {distance_meters: 9.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
134 CREATE (roomB12)-[:CONNECTS_TO {distance_meters: 9.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
135 false}]->(corrB1)
136 CREATE (qrB1)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
137 CREATE (liftB)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
138 true}]->(qrB1)
139 CREATE (corrB1)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
140 CREATE (stairsB)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
141 false}]->(corrB1)
142 // Second Floor Connections
143 CREATE (qrB2)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
144 CREATE (roomB22)-[:CONNECTS_TO {distance_meters: 10.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
145 false}]->(qrB2)
146 CREATE (corrB2)-[:CONNECTS_TO {distance_meters: 14.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
147 CREATE (roomB21)-[:CONNECTS_TO {distance_meters: 14.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
148 false}]->(corrB2)
149 CREATE (corrB2)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
150 CREATE (roomB23)-[:CONNECTS_TO {distance_meters: 8.0, wheelchair_accessible: true, tactile_paving: false, stairs: false, lift:
151 false}]->(corrB2)
152 CREATE (qrB2)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
153 CREATE (liftB)-[:CONNECTS_TO {distance_meters: 4.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
154 true}]->(qrB2)
155 CREATE (corrB2)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
156 CREATE (stairsB)-[:CONNECTS_TO {distance_meters: 6.0, wheelchair_accessible: false, tactile_paving: false, stairs: true, lift:
157 false}]->(corrB2)
158 // --- INTER-BUILDING PATHS ---
159 CREATE (entA)-[:CONNECTS_TO {distance_meters: 50.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
160 CREATE (entB)-[:CONNECTS_TO {distance_meters: 50.0, wheelchair_accessible: true, tactile_paving: true, stairs: false, lift:
    false}]->(entA)
```

Frontend — Services & Types

api.ts TypeScript | frontend/src/services/api.ts

```typescript
/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */

import { NavigationMode, RouteOption, HazardIssue } from '../types';

// Base URL for the backend API.
const API_BASE_URL = (import.meta.env.VITE_API_URL || '').replace(/\/$/, '');

// ■■■ Agent Query Types ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

export interface AgentStep {
 tool: string;
 input: string;
 output: string;
}

export interface NavigateResponse {
 response: string;
 route_data: any | null;
 agent_steps: AgentStep[];
 intent: string | null;
 profile_applied: string | null;
}

/**
 * Send a natural-language query to the CampusPilot agent backend.
 * Returns the LLM response, structured route data, and real tool traces.
 * Falls back to offline route simulation when the backend is unreachable.
 */
export async function queryAgent(
 origin: string,
 destination: string,
 profile: NavigationMode,
 qr_location?: string
): Promise<NavigateResponse> {
 if (API_BASE_URL) {
 try {
 const res = await fetch(`${API_BASE_URL}/navigate/`, {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ origin, destination, profile, qr_location })
 });
 if (res.ok) return await res.json();
 } catch (err) {
 // Backend unavailable, fallback to offline calculation
 }
 }

 // Offline fallback — calculate route locally
 const routes = await getRoute(origin, destination, profile);
 return {
 response: routes[0]?.features?.[0] || 'Route calculated offline.',
 route_data: null,
 agent_steps: [],
 intent: 'navigate',
 profile_applied: profile
 };
}

/**
 * Fetch a route from origin to destination tailored to accessibility profiles.
 * Tries the backend first. Falls back to a rich offline route calculator.
 */
export async function getRoute(
 origin: string,
 destination: string,
 profile: NavigationMode
): Promise<RouteOption[]> {
 const cacheKey = `route_${origin}_${destination}_${profile}`.toLowerCase();

 if (API_BASE_URL) {
 try {
 const response = await fetch(`${API_BASE_URL}/navigate`, {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ origin, destination, profile })
```

```
});
if (response.ok) {
const data = await response.json();
localStorage.setItem(cacheKey, JSON.stringify(data));
return data;
}
} catch (err) {
// Backend unavailable, fallback to cache or offline calculation
}
}

// Serve from offline cache if available.
const cached = localStorage.getItem(cacheKey);
if (cached) {
return JSON.parse(cached);
}

// Offline route calculation — profile-aware fallback when backend is unreachable.

// Generate simulated route options based on profile constraints
const minutes = Math.floor(Math.random() * 6) + 4;
const miles = parseFloat((0.2 + Math.random() * 0.3).toFixed(2));

const options: RouteOption[] = [];

if (profile === 'wheelchair') {
options.push({
id: 'opt-1',
name: 'Step-Free Ramp Route',
estMinutes: minutes,
distanceMiles: miles,
isOptimal: true,
isAccessible: true,
features: ['Elevators', 'Ramps', 'Extra-wide corridors'],
warnings: ['Heavy doors at main corridor B']
});
options.push({
id: 'opt-2',
name: 'Elevator-Only Service Route',
estMinutes: minutes + 3,
distanceMiles: miles + 0.1,
isOptimal: false,
isAccessible: true,
features: ['Elevators', 'Automatic push-doors'],
warnings: ['Slightly longer distance']
});
} else if (profile === 'vision') {
options.push({
id: 'opt-1',
name: 'Tactile Navigation Walkway',
estMinutes: minutes + 1,
distanceMiles: miles,
isOptimal: true,
isAccessible: true,
features: ['Tactile guidance lines', 'Sound beacons active', 'Continuous handrails'],
warnings: ['Wet floor reported in Corridor B']
});
} else if (profile === 'chronic') {
options.push({
id: 'opt-1',
name: 'Rest-Bench Friendly Path',
estMinutes: minutes + 2,
distanceMiles: miles,
isOptimal: true,
isAccessible: true,
features: ['Frequent bench seating', 'Low-elevation change', 'Air conditioned corridors'],
warnings: []
});
} else {
// Standard and others
options.push({
id: 'opt-1',
name: 'Direct Central Stairs Route',
estMinutes: minutes - 2,
distanceMiles: miles - 0.05,
isOptimal: true,
isAccessible: profile === 'standard',
features: ['Direct stairwells', 'Central walkway'],
warnings: ['Contains 2 sets of stairs']
});
options.push({
id: 'opt-2',
name: 'Accessible Outer Bypass',
```

```
 estMinutes: minutes + 1,
 distanceMiles: miles + 0.05,
 isOptimal: false,
 isAccessible: true,
 features: ['Step-free outer path', 'Ramps'],
 warnings: []
 });
 }

 // Cache final result
 localStorage.setItem(cacheKey, JSON.stringify(options));
 return options;
}

/**
 * Look up a scanned QR checkpoint's metadata.
 * Fetches the local JSON file.
 */
export async function lookupQR(code: string): Promise<any> {
 if (API_BASE_URL) {
 try {
 const response = await fetch(`${API_BASE_URL}/qr/lookup?code=${encodeURIComponent(code)}`);
 if (response.ok) {
 const data = await response.json();
 unlockBuildingMap(data.building);
 return data;
 }
 } catch (err) {
 // Backend unavailable, fallback to local
 }
 }

 // Fetch local checkpoint list
 try {
 const response = await fetch('/qr_map.json');
 if (response.ok) {
 const map = await response.json();
 const match = map[code];
 if (match) {
 unlockBuildingMap(match.building);
 return match;
 }
 }
 } catch (err) {
 // Local file not available
 }

 throw new Error('QR Checkpoint code not recognized or invalid.');
}

/**
 * Report a new hazard/obstacle to the map network.
 */
export async function reportIncident(
 incident: Omit<HazardIssue, 'id' | 'timestamp'>
): Promise<HazardIssue> {
 const mockResult: HazardIssue = {
 ...incident,
 id: `issue-${Math.floor(Math.random() * 10000)}`,
 timestamp: new Date().toISOString(),
 isCustom: true
 };

 if (API_BASE_URL) {
 try {
 const response = await fetch(`${API_BASE_URL}/alert`, {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify(mockResult)
 });
 if (response.ok) {
 return await response.json();
 }
 } catch (err) {
 // Backend unavailable, persist locally
 }
 }

 // Persist locally so the report survives a page reload.
 const existing = localStorage.getItem('custom_issues');
 const issues: HazardIssue[] = existing ? JSON.parse(existing) : [];
 issues.push(mockResult);
 localStorage.setItem('custom_issues', JSON.stringify(issues));
```

```
 return mockResult;
}

/**
 * Unlock a building map in the device's persistent cache.
 * Storing this in local storage ensures users do not have to rescan QR codes.
 */
export function unlockBuildingMap(buildingName: string): void {
 const unlocked = getUnlockedBuildings();
 if (!unlocked.includes(buildingName)) {
 unlocked.push(buildingName);
 localStorage.setItem('unlocked_buildings', JSON.stringify(unlocked));
 }
}

/**
 * Check if a building's indoor map is already cached/unlocked on the device.
 */
export function isBuildingMapUnlocked(buildingName: string): boolean {
 return getUnlockedBuildings().includes(buildingName);
}

/**
 * Get all unlocked buildings from storage.
 */
export function getUnlockedBuildings(): string[] {
 const cached = localStorage.getItem('unlocked_buildings');
 return cached ? JSON.parse(cached) : ['Engineering Block A'];
}

/**
 * Upload an image (File or base64 data URL string) to Cloudinary.
 * Uses direct client-side unsigned uploads.
 */
export async function uploadToCloudinary(fileOrDataUrl: File | string): Promise<string> {
 const cloudName = import.meta.env.VITE_CLOUDINARY_CLOUD_NAME;
 const uploadPreset = import.meta.env.VITE_CLOUDINARY_UPLOAD_PRESET;

 if (!cloudName || !uploadPreset || cloudName === 'your_cloud_name' || uploadPreset === 'your_unsigned_preset') {
 // Cloudinary not configured — return a local object URL as fallback.
 return typeof fileOrDataUrl === 'string' ? fileOrDataUrl : URL.createObjectURL(fileOrDataUrl);
 }

 const formData = new FormData();
 formData.append('file', fileOrDataUrl);
 formData.append('upload_preset', uploadPreset);

 const response = await fetch(`https://api.cloudinary.com/v1_1/${cloudName}/image/upload`, {
 method: 'POST',
 body: formData
 });

 if (!response.ok) {
 const errorData = await response.json();
 throw new Error(errorData.error?.message || 'Failed to upload image to Cloudinary.');
 }

 const result = await response.json();
 return result.secure_url;
}
```

types.ts TypeScript | frontend/src/types.ts

```
/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */

export type NavigationMode = 'standard' | 'wheelchair' | 'vision' | 'hearing' | 'cognitive' | 'chronic';

export interface AccessibilityProfile {
 id: NavigationMode;
 name: string;
 description: string;
 icon: string;
 details: string;
 color: string;
 accentColor: string;
}

export interface SavedMap {
 id: string;
```

```
 name: string;
 floorsCount: number;
 sizeMb: number;
 imageThumbnail: string;
 category: 'academic' | 'facility' | 'other';
}

export interface HazardIssue {
 id: string;
 type: 'construction' | 'lift' | 'toilet' | 'wet_floor' | 'door' | 'other';
 typeName: string;
 location: string;
 details: string;
 photoAttached: boolean;
 photoUrl?: string;
 timestamp: string;
 isCustom?: boolean;
}

export interface RouteOption {
 id: string;
 name: string;
 estMinutes: number;
 distanceMiles: number;
 isOptimal: boolean;
 isAccessible: boolean;
 features: string[];
 warnings: string[];
}

export interface NavStep {
 stepIndex: number;
 instruction: string;
 subInstructions: string;
 directionIcon: 'straight' | 'turn_left' | 'turn_right' | 'elevator' | 'stairs' | 'warning';
 distanceAhead: string;
}
```

Frontend — Core Components

NavigationChat.tsx

TypeScript/React | frontend/src/components/NavigationChat.tsx

```tsx
/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */

import React, { useState, useEffect, useRef } from 'react';
import { Send, Mic, Cpu, Sparkles, Navigation, Volume2, AlertTriangle, Compass, Bot, CheckCircle, X } from 'lucide-react';
import { NavigationMode, RouteOption } from '../types';
import { queryAgent } from '../services/api';

interface Message {
 id: string;
 sender: 'user' | 'agent';
 text: string;
 timestamp: Date;
 agentSteps?: {
 stepName: string;
 details: string;
 status: 'pending' | 'success' | 'info';
 }[];
 actionButton?: {
 label: string;
 onClick: () => void;
 };
}

interface NavigationChatProps {
 currentProfileMode: NavigationMode;
 currentLocation: string;
 lastQrLocation?: string;
 onNavigateToDestination: (origin: string, destination: string) => void;
 onClose?: () => void;
}

export default function NavigationChat({
 currentProfileMode,
 currentLocation,
 lastQrLocation,
 onNavigateToDestination,
 onClose
}: NavigationChatProps) {
 const [messages, setMessages] = useState<Message[]>([
 {
 id: 'msg-init-1',
 sender: 'agent',
 text: `CampusPilot AI Navigator ready.\n\nActive profile: ${currentProfileMode}\nCurrent location: ${currentLocation}\n\nWhere would you li…
 timestamp: new Date()
 }
 ]);

 const [inputValue, setInputValue] = useState('');
 const [isVoiceListening, setIsVoiceListening] = useState(false);
 const [isAgentTyping, setIsAgentTyping] = useState(false);
 const chatEndRef = useRef<HTMLDivElement>(null);

 // Auto scroll to bottom
 useEffect(() => {
 chatEndRef.current?.scrollIntoView({ behavior: 'smooth' });
 }, [messages, isAgentTyping]);

 const handleSendText = async (text: string) => {
 const trimmed = text.trim();
 if (!trimmed) return;

 // Add user message
 const userMsg: Message = {
 id: `msg-user-${Date.now()}`,
 sender: 'user',
 text: trimmed,
 timestamp: new Date()
 };

 setMessages(prev => [...prev, userMsg]);
 setInputValue('');
 setIsAgentTyping(true);

 try {
 // Call the real backend agent
```

```
  const result = await queryAgent(
  currentLocation,
  trimmed,
  currentProfileMode,
  lastQrLocation
  );

  // Map real agent steps to display format
  const agentSteps: Message['agentSteps'] = result.agent_steps.map((step, i) => ({
  stepName: `${i + 1}. Tool: ${step.tool}`,
  details: `Input: ${step.input}\nOutput: ${step.output}`,
  status: 'success' as const
  }));

  // Add intent step at the front if available
  if (result.intent) {
  agentSteps.unshift({
  stepName: '0. Intent Classified',
details: `Intent: [${result.intent.toUpperCase()}] | Profile: [${result.profile_applied?.toUpperCase() ||
currentProfileMode.toUpperCas…
  });
  }

  let responseText = result.response;
  if (result.intent === 'emergency') {
  // Emergency responses are pre-LLM — flag them distinctly
  agentSteps.length = 0;
  agentSteps.push({
  stepName: 'Emergency Override',
  details: 'Pre-LLM heuristic triggered. Agent bypassed. Response time < 1ms.',
  status: 'info' as const
  });
  }

  const agentMsg: Message = {
  id: `msg-agent-${Date.now()}`,
  sender: 'agent',
  text: responseText,
  timestamp: new Date(),
  agentSteps,
  actionButton: result.route_data ? {
  label: 'Start Guidance',
  onClick: () => onNavigateToDestination(currentLocation, trimmed)
  } : undefined
  };

  setMessages(prev => [...prev, agentMsg]);

  } catch (err) {
  setMessages(prev => [...prev, {
  id: `msg-err-${Date.now()}`,
  sender: 'agent',
  text: 'Unable to reach the navigation service. Please check your connection and try again.',
  timestamp: new Date()
  }]);
  } finally {
  setIsAgentTyping(false);
  }
  };

  return (
  <div className="flex flex-col bg-zinc-950 text-white h-[calc(100vh-76px)] overflow-hidden font-sans relative">
  {/* ■■■ HUD HEADER ■■■ */}
  <header className="px-5 py-3.5 bg-zinc-900 border-b border-zinc-800/80 flex items-center justify-between shrink-0 z-10">
  <div className="flex items-center gap-2">
  <Bot className="w-5 h-5 text-teal-400 animate-pulse" />
  <div className="text-left">
  <h2 className="text-sm font-extrabold text-white tracking-wide leading-none">CampusPilot AI Navigator</h2>
  <span className="text-[10px] text-teal-400 font-mono tracking-widest uppercase mt-1 block">Agentic ReAct — Live</span>
  </div>
  </div>
  {onClose && (
  <button
  onClick={onClose}
  className="text-zinc-400 hover:text-white p-1.5 rounded-xl hover:bg-zinc-800 transition-colors cursor-pointer"
  aria-label="Close Chat"
  >
  <X className="w-5 h-5" />
  </button>
  )}
  </header>

  {/* ■■■ CHAT BUBBLE STREAM ■■■ */}
```

```
<div className="flex-1 overflow-y-auto px-5 py-4 space-y-6 no-scrollbar pb-32 z-0">
{messages.map((msg) => (
<div
key={msg.id}
className={`flex flex-col max-w-[85%] ${
msg.sender === 'user' ? 'ml-auto items-end' : 'mr-auto items-start'
}`}
>
{/* Sender Label */}
<span className="text-[10px] text-zinc-500 font-mono mb-1 select-none">
{msg.sender === 'user' ? 'YOU' : 'CAMPUSPILOT_AGENT'}
</span>

{/* Bubble */}
<div
className={`rounded-2xl p-4 text-sm font-medium leading-relaxed shadow-md select-text ${
msg.sender === 'user'
? 'bg-teal-600 text-white rounded-tr-none'
: 'bg-zinc-900 text-zinc-105 border border-zinc-850 rounded-tl-none space-y-3'
}`}
>
{/* Message text */}
<div className="whitespace-pre-wrap text-left font-sans prose prose-invert prose-sm">
{msg.text}
</div>

{/* Agent step trace — shows real tool calls from LangChain */}
{msg.agentSteps && msg.agentSteps.length > 0 && (
<div className="mt-3 space-y-1.5 border-t border-zinc-800 pt-3">
<span className="text-[9px] font-mono text-zinc-600 tracking-widest uppercase block mb-1">Agent Trace</span>
{msg.agentSteps.map((step, i) => (
<div key={i} className="flex items-start gap-2 text-[11px]">
<span className={`mt-0.5 w-2 h-2 rounded-full shrink-0 ${
step.status === 'success' ? 'bg-teal-400' :
step.status === 'info' ? 'bg-blue-400' : 'bg-zinc-500'
}`} />
<div>
<span className="font-bold text-zinc-300">{step.stepName}</span>
<span className="text-zinc-500 ml-1 break-all">{step.details}</span>
</div>
</div>
))}
</div>
)}

{/* Action Button inside message */}
{msg.actionButton && (
<div className="mt-4 pt-2">
<button
onClick={msg.actionButton.onClick}
className="w-full bg-[#60f8cb] text-zinc-950 hover:bg-[#4edab0] font-bold h-11 rounded-xl flex items-center justify-center ga…
>
<Navigation className="w-4 h-4 rotate-45" />
{msg.actionButton.label} </button>
</div>
)}
</div>
</div>
))}

{/* Typing placeholder animation */}
{isAgentTyping && (
<div className="flex flex-col items-start mr-auto max-w-[80%]">
<span className="text-[10px] text-zinc-500 font-mono mb-1">CAMPUSPILOT_AGENT</span>
<div className="bg-zinc-900 border border-zinc-850 p-4 rounded-2xl rounded-tl-none flex items-center gap-2">
<div className="h-2 w-2 bg-teal-400 rounded-full animate-bounce" style={{ animationDelay: '0ms' }} />
<div className="h-2 w-2 bg-teal-400 rounded-full animate-bounce" style={{ animationDelay: '150ms' }} />
<div className="h-2 w-2 bg-teal-400 rounded-full animate-bounce" style={{ animationDelay: '300ms' }} />
</div>
</div>
)}

<div ref={chatEndRef} />
</div>

{/* ■■■ BOTTOM CONTROL SHEETS (INPUT BAR) ■■■ */}
<footer className="absolute bottom-0 left-0 right-0 bg-zinc-900 border-t border-zinc-850 px-4 pt-3 pb-6 shrink-0 z-10">
<form
onSubmit={(e) => {
e.preventDefault();
handleSendText(inputValue);
}}
className="relative flex items-center w-full max-w-xl mx-auto"
```

```
 >
 {/* Text Input */}
 <div className="relative flex-1">
 <input
 type="text"
 placeholder="Ask CampusPilot Navigator..."
 value={inputValue}
 onChange={(e) => setInputValue(e.target.value)}
className="block w-full bg-zinc-950 border border-zinc-800 text-white rounded-xl py-3.5 pl-4 pr-12 text-sm font-semibold
fóeus:outl…

 {/* Send button */}
 <button
 type="submit"
 disabled={!inputValue.trim()}
className="absolute inset-y-0 right-2 pr-3 flex items-center text-teal-400 hover:text-teal-300 disabled:text-zinc-650
cursor-pointer"
 <Send className="w-4 h-4" />
 </button>
 </div>
 </form>
 </footer>
 </div>
 );
}
```

RouteOptionsView.tsx TypeScript/React | frontend/src/components/RouteOptionsView.tsx

```
/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */

import React, { useState, useEffect } from 'react';
import { ArrowLeft, Clock, CheckCircle2, ShieldAlert, Coffee, HelpCircle, Navigation, Info, User, Check, Footprints,
ArrowUpDown, Navigation2 } from 'lucide-react';
import type { NavigationMode, RouteOption } from '../types';
import { getRoute } from '../services/api';

interface RouteOptionsViewProps {
 onBack: () => void;
 onStartRoute: (routeId: string) => void;
 originName?: string;
 destName?: string;
 currentProfileMode?: NavigationMode;
 themeMode?: 'light' | 'dark';
}

export default function RouteOptionsView({
 onBack,
 onStartRoute,
 originName = 'Library',
 destName = 'Science Hall',
 currentProfileMode = 'standard',
 themeMode = 'light'
}: RouteOptionsViewProps) {
 const isWheelchairMode = currentProfileMode === 'wheelchair';
 const [selectedRouteId, setSelectedRouteId] = useState<string>(
 isWheelchairMode ? 'wheelchair-safe' : 'route-optimal'
 );
 const [assistantMessage, setAssistantMessage] = useState<string>('');

 // Fetch real routes from backend
 const [routes, setRoutes] = useState<RouteOption[]>([]);
 const [loading, setLoading] = useState(true);

 useEffect(() => {
 setLoading(true);
 getRoute(originName, destName, currentProfileMode)
 .then(data => {
 setRoutes(data);
 if (data.length > 0) {
 setSelectedRouteId(data[0].id);
 }
 })
 .catch(() => {
 // Route fetch failed, display fallback routes
 })
 .finally(() => setLoading(false));
 }, [originName, destName, currentProfileMode]);

 const handleAssistantQuery = (query: string) => {
 switch (query) {
 case 'coffee':
```

```
setAssistantMessage('The Hub Cafe is located inside Engineering Block A Lobby, directly along your route. The counter is fully ramped.');
break;
case 'toilet':
setAssistantMessage('An accessible, gender-neutral restroom is located on the Ground Floor of Engineering Block A, adjacent to Lift 2.');
break;
case 'stairs':
setAssistantMessage('The Wheelchair Safe Route has been updated to bypass stairs and prioritise the Lift 2 corridor entrance.');
break;
case 'lift':
setAssistantMessage('Lift 2 is operational. Lift 1 is currently undergoing scheduled maintenance.');
break;
case 'obstacle':
setAssistantMessage('Obstacle report submitted. Facility staff have been notified of the reported hazard in Corridor B.');
break;
default:
setAssistantMessage('');
}
};

return (
<div className="w-full max-w-md mx-auto px-4 pt-4 pb-32 font-sans select-none min-h-screen">
{/* Dynamic Header */}
{isWheelchairMode ? (
<header className="flex justify-between items-center w-full mb-6 pb-3 border-b border-zinc-200">
<div className="flex items-center gap-3">
<button
aria-label="Go Back"
onClick={onBack}
className="p-1.5 hover:bg-zinc-200 rounded-full transition-colors"
>
<ArrowLeft className="w-5 h-5 text-zinc-850" />
</button>
<h1 className="text-xl font-black text-[#0f2942]">
Route to Room 204
</h1>
</div>
<button className="w-8 h-8 rounded-full border border-zinc-350 flex items-center justify-center text-zinc-650 hover:text-zinc-800 transition-colors">
<X className="w-4 h-4" />
</button>
</header>
) : (
<header className="flex justify-between items-center w-full mb-6 pb-3 border-b border-zinc-200">
<div className="flex items-center gap-3">
<button
aria-label="Go Back"
onClick={onBack}
className="p-1.5 hover:bg-zinc-200 rounded-full transition-colors"
>
<ArrowLeft className="w-5 h-5 text-zinc-850" />
</button>
<h1 className="text-lg font-bold text-zinc-900 truncate max-w-[280px]">
{originName} to {destName}
</h1>
</div>
</header>
)}

{/* Assistant dynamic notification overlay */}
{assistantMessage && (
<div className="mb-4 bg-blue-50 border border-blue-250 rounded-xl p-3 flex items-start gap-2.5 shadow-sm relative">
<div className="p-1.5 bg-blue-100 rounded-lg text-blue-800">
<Info className="w-4 h-4 shrink-0" />
</div>
<div className="flex-1">
<p className="text-xs font-bold text-zinc-800">Assistant Note</p>
<p className="text-xs text-zinc-650 mt-0.5 leading-relaxed">
{assistantMessage}
</p>
</div>
<button
onClick={() => setAssistantMessage('')}
className="text-zinc-400 hover:text-zinc-600 text-[10px] font-bold px-1.5 py-0.5"
>
Close
</button>
</div>
)}

{/* Available Routes or Route Options title */}
<h2 className="text-base font-extrabold text-zinc-800 mb-1">
{isWheelchairMode ? 'Available Routes' : 'Route Options'}
</h2>
{!loading && routes.length > 0 && (
```

```
 <p className="text-[10px] font-mono text-zinc-400 mb-4">
 {routes.length} route{routes.length !== 1 ? 's' : ''} available
 </p>
 )}

 {/* Loading state */}
 {loading && (
 <div className="flex flex-col items-center justify-center py-16 gap-3">
 <Loader2 className="w-8 h-8 text-teal-500 animate-spin" />
 <p className="text-xs font-bold text-zinc-400">Fetching routes from agent...</p>
 </div>
 )}

 {!loading && isWheelchairMode ? (
 /* WHEELCHAIR MODE LAYOUT (Light Mode screenshot sample) */
 <div className="space-y-4">
 {/* Card 1: Wheelchair Safe Route (Selected) */}
 <div
 onClick={() => setSelectedRouteId('wheelchair-safe')}
 className={`border rounded-2xl p-4 transition-all relative overflow-hidden flex flex-col gap-3 ${
 selectedRouteId === 'wheelchair-safe'
 ? 'bg-white border-emerald-500 shadow-[0_0_15px_rgba(0,196,154,0.1)]'
 : 'bg-white border-zinc-200 opacity-80'
 }`}
 >
 {selectedRouteId === 'wheelchair-safe' && (
 <div className="absolute left-0 top-0 bottom-0 w-1 bg-emerald-500" />
 )}

 <div className="flex justify-between items-start">
 <div className="flex gap-3">
 {/* Left wheelchair icon circle */}
 <div className="w-10 h-10 rounded-full bg-emerald-100 text-emerald-600 flex items-center justify-center shrink-0">
 <CheckCircle2 className="w-5 h-5" />
 </div>
 <div>
 <h3 className="font-extrabold text-zinc-900 text-sm">
 Wheelchair Safe Route
 </h3>
 <div className="flex items-center gap-1 text-[11px] text-emerald-650 font-bold mt-0.5">
 <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500" />
 <span>Optimal Path</span>
 </div>
 </div>
 </div>

 {/* Right duration badge */}
 <div className="bg-[#f0effa] text-zinc-850 text-xs font-bold py-1.5 px-3 rounded-full flex items-center gap-1 shrink-0">
 <Clock className="w-3.5 h-3.5 text-zinc-500" />
 <span>{routes[0]?.estMinutes || 4} min</span>
 </div>
 </div>

 {/* Badges */}
 <div className="flex flex-wrap gap-1.5">
 <span className="bg-zinc-50 border border-zinc-200 text-zinc-650 text-[10px] font-bold px-2.5 py-1 rounded-full flex
itAmsowUDown className="w-3.5 h-3.5 text-zinc-500" />
 Uses Lift 2
 </span>
 <span className="bg-zinc-50 border border-zinc-200 text-zinc-650 text-[10px] font-bold px-2.5 py-1 rounded-full flex
itEmotpoints className="w-3.5 h-3.5 text-zinc-500" />
 Step-free
 </span>
 </div>

 {/* Buttons inside selected Card 1 */}
 {selectedRouteId === 'wheelchair-safe' && (
 <div className="flex flex-col gap-2 mt-2">
 <button
 onClick={(e) => {
 e.stopPropagation();
 onStartRoute('wheelchair-safe');
 }}
className="bg-[#006d51] hover:bg-[#005740] text-white font-bold text-xs py-3 px-4 rounded-xl flex items-center justify-center
g>
 <Navigation className="w-3.5 h-3.5 fill-current" />
 Start Guidance
 </button>
 <button
 onClick={(e) => {
 e.stopPropagation();
 setAssistantMessage('Map preview loaded. Elevator access is highlighted on the floor plan.');
 }}
```

```
className="bg-transparent border border-[#006d51] text-[#006d51] hover:bg-teal-50 font-bold text-xs py-3 px-4 rounded-xl flex i…
 <Map className="w-3.5 h-3.5" />
 Show Map Preview
 </button>
 </div>
 )}
 </div>

 {/* Card 2: Standard Route (Stairs warning, not selected) */}
 <div
 onClick={() => setSelectedRouteId('standard-route')}
 className={`border rounded-2xl p-4 transition-all relative overflow-hidden flex flex-col gap-3 ${
 selectedRouteId === 'standard-route'
 ? 'bg-white border-emerald-500 shadow-[0_0_15px_rgba(0,196,154,0.1)]'
 : 'bg-white border-zinc-200 opacity-80'
 }`}
 >
 {selectedRouteId === 'standard-route' && (
 <div className="absolute left-0 top-0 bottom-0 w-1 bg-emerald-500" />
 )}

 <div className="flex justify-between items-start">
 <div className="flex gap-3">
 {/* Left walking icon circle */}
 <div className="w-10 h-10 rounded-full bg-zinc-100 text-zinc-500 flex items-center justify-center shrink-0">
 <Footprints className="w-5 h-5" />
 </div>
 <div>
 <h3 className="font-extrabold text-zinc-900 text-sm">
 Standard Route
 </h3>
 <div className="bg-red-50 text-red-650 border border-red-500/10 text-[10px] font-bold px-2 py-0.5 rounded mt-1 inline-flex i…
 <ShieldAlert className="w-3.5 h-3.5" />
 Involves Stairs
 </div>
 </div>
 </div>

 {/* Right duration badge */}
 <div className="bg-[#f0effa] text-zinc-850 text-xs font-bold py-1.5 px-3 rounded-full flex items-center gap-1 shrink-0">
 <Clock className="w-3.5 h-3.5 text-zinc-500" />
 <span>{routes[1]?.estMinutes || 2} min</span>
 </div>
 </div>

 {/* If selected standard route */}
 {selectedRouteId === 'standard-route' && (
 <div className="flex flex-col gap-2 mt-2">
 <button
 onClick={(e) => {
 e.stopPropagation();
 onStartRoute('standard-route');
 }}
 className="bg-primary hover:bg-[#002f5c] text-white font-bold text-xs py-3 px-4 rounded-xl flex items-center justify-center gap…
 <Navigation className="w-3.5 h-3.5" />
 Start Guidance
 </button>
 </div>
 )}
 </div>

 {/* Quick Assistance Section */}
 <section className="pt-4">
 <h3 className="text-xs font-extrabold text-zinc-450 uppercase tracking-widest mb-3">
 Quick Assistance
 </h3>
 <div className="flex flex-col gap-2">
 <button
 onClick={() => handleAssistantQuery('lift')}
 className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-3 px-4 rounded-full flex items…
 <HelpCircle className="w-4 h-4 text-blue-500 shrink-0" />
 Where is nearest lift?
 </button>
 <button
 onClick={() => handleAssistantQuery('obstacle')}
 className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-3 px-4 rounded-full flex items…
 <AlertTriangle className="w-4 h-4 text-red-500 shrink-0" />
 Report obstacle
 </button>
```

```
<button
onClick={() => handleAssistantQuery('toilet')}
className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-3 px-4 rounded-full flex
items…
>
<span className="text-teal-500 shrink-0 text-sm">
<CheckCircle2 className="w-4 h-4" />
</span>
Find accessible toilet
</button>
</div>
</section>
</div>
) : (
/* STANDARD MODE LAYOUT (Dark Mode screenshot sample) */
<div className="space-y-4">
{/* Card 1: Most Direct (Selected) */}
<div
onClick={() => setSelectedRouteId('route-optimal')}
className={`border rounded-2xl p-4 transition-all relative overflow-hidden flex flex-col gap-3 ${
selectedRouteId === 'route-optimal'
? 'bg-white border-teal-500 shadow-[0_0_15px_rgba(0,196,154,0.1)]'
: 'bg-white border-zinc-200 opacity-80'
}`}
>
{selectedRouteId === 'route-optimal' && (
<div className="absolute left-0 top-0 bottom-0 w-1 bg-teal-500" />
)}

<div className="flex justify-between items-start">
<div className="flex gap-3">
<div className="w-10 h-10 rounded-full bg-emerald-50 text-emerald-600 flex items-center justify-center shrink-0">
<Navigation className="w-5 h-5 rotate-45" />
</div>
<div>
<h3 className="font-extrabold text-zinc-900 text-sm">
Most Direct
</h3>
<p className="text-xs text-zinc-500 mt-1 font-semibold">
Est. {routes[0]?.estMinutes || 5} mins • {routes[0]?.distanceMiles || 0.2} miles
</p>
</div>
</div>

{/* Selected badge */}
<span className="bg-emerald-500 text-white border border-emerald-500/10 font-sans font-bold text-[10px] px-2.5 py-1
rounded-full…
<Check className="w-3.5 h-3.5 stroke-[3px]" /> Selected
</span>
</div>

{/* Button inside Card 1 */}
{selectedRouteId === 'route-optimal' && (
<div className="mt-2">
<button
onClick={(e) => {
e.stopPropagation();
onStartRoute('route-optimal');
}}
className={`w-full py-3 px-4 rounded-xl font-sans font-bold text-xs flex items-center justify-center gap-2 shadow-sm
transition…
themeMode === 'light'
? 'bg-[#002f5c] hover:bg-[#002447]'
: 'bg-[#1b3a5f] hover:bg-[#152e4c]'
}`}
>
<Navigation className="w-3.5 h-3.5" />
Start Navigation
</button>
</div>
)}
</div>

{/* Card 2: Accessible Route */}
<div
onClick={() => setSelectedRouteId('route-accessible')}
className={`border rounded-2xl p-4 transition-all relative overflow-hidden flex flex-col gap-3 ${
selectedRouteId === 'route-accessible'
? 'bg-white border-teal-500 shadow-[0_0_15px_rgba(0,196,154,0.1)]'
: 'bg-white border-zinc-200 opacity-80'
}`}
>
{selectedRouteId === 'route-accessible' && (
<div className="absolute left-0 top-0 bottom-0 w-1 bg-teal-500" />
)}
```

```
 <div className="flex justify-between items-start">
 <div className="flex gap-3">
 <div className="w-10 h-10 rounded-full bg-zinc-150 text-zinc-550 flex items-center justify-center shrink-0">
 <Navigation className="w-5 h-5 rotate-45" />
 </div>
 <div>
 <h3 className="font-extrabold text-zinc-900 text-sm">
 Accessible Route
 </h3>
 <p className="text-xs text-zinc-500 mt-1 font-semibold">
 Est. {routes[1]?.estMinutes || 8} mins • {routes[1]?.distanceMiles || 0.3} miles
 </p>
 </div>
 </div>
 </div>

 {/* Badges */}
 <div className="flex flex-wrap gap-1.5 mt-1">
 <span className="bg-zinc-50 border border-zinc-200 text-zinc-650 text-[10px] font-bold px-2.5 py-1 rounded-full">
 Elevators
 </span>
 <span className="bg-zinc-50 border border-zinc-200 text-zinc-650 text-[10px] font-bold px-2.5 py-1 rounded-full">
 Ramps
 </span>
 </div>

 {selectedRouteId === 'route-accessible' && (
 <div className="mt-2">
 <button
 onClick={(e) => {
 e.stopPropagation();
 onStartRoute('route-accessible');
 }}
className={`w-full py-3 px-4 rounded-xl font-sans font-bold text-xs flex items-center justify-center gap-2 shadow-sm
transition…
 themeMode === 'light'
 ? 'bg-[#002f5c] hover:bg-[#002447]'
 : 'bg-[#1b3a5f] hover:bg-[#152e4c]'
 }`}
 >
 <Navigation className="w-3.5 h-3.5" />
 Start Navigation
 </button>
 </div>
 )}
 </div>

 {/* Ask Assistant Section */}
 <section className="pt-4">
 <h3 className="text-xs font-extrabold text-zinc-450 uppercase tracking-widest mb-3">
 Ask Assistant
 </h3>
 <div className="flex flex-wrap gap-2">
 <button
 onClick={() => handleAssistantQuery('coffee')}
className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-2.5 px-4 rounded-full flex
ite…
 >
 <Coffee className="w-4 h-4 text-emerald-500" />
 Coffee on route?
 </button>
 <button
 onClick={() => handleAssistantQuery('toilet')}
className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-2.5 px-4 rounded-full flex
ite…
 >
 <HelpCircle className="w-4 h-4 text-blue-500" />
 Nearest restroom?
 </button>
 <button
 onClick={() => handleAssistantQuery('stairs')}
className="bg-white hover:bg-zinc-50 border border-zinc-200 text-zinc-700 font-semibold text-xs py-2.5 px-4 rounded-full flex
ite…
 >
 <ShieldAlert className="w-4 h-4 text-amber-500" />
 Avoid stairs
 </button>
 </div>
 </section>
 </div>
 )}
 </div>
 );
}
```

SosAlertView.tsx

TypeScript/React | frontend/src/components/SosAlertView.tsx

```
/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */

import React, { useState, useEffect } from 'react';
import { AlertTriangle, Send, Bluetooth, Wifi, MapPin, MessageSquare } from 'lucide-react';

interface SosAlertViewProps {
 onCancel: () => void;
 lastKnownLocation?: string;
}

export default function SosAlertView({
 onCancel,
 lastKnownLocation = 'Engineering Block A — Ground Floor Entrance'
}: SosAlertViewProps) {
 const [statusMessage, setStatusMessage] = useState('');

 const handleSendSms = () => {
 setStatusMessage('Initiating emergency alert transmission to Campus Security...');
 setTimeout(() => setStatusMessage('Emergency alert submitted. Campus Security has been notified.'), 1500);
 };

 const handleBroadcastMesh = () => {
 setStatusMessage('Initiating emergency beacon broadcast to connected devices...');
 setTimeout(() => setStatusMessage('Emergency beacon has been broadcast to connected campus devices.'), 1500);
 };

 return (
 <div className="fixed inset-0 bg-[#0d0000] text-white flex flex-col font-sans select-none overflow-y-auto pb-12 z-[9999]
justify-between">
 {/* Background grid overlay */}
 <div
 className="absolute inset-0 opacity-[0.02] pointer-events-none"
 style={{
backgroundImage: `url("data:image/svg+xml,%3Csvg viewBox='0 0 200 200' xmlns='http://www.w3.org/2000/svg'%3E%3Cfilter
id='noise'%3E%3Cf…
 }}
 />

 {/* Header */}
 <header className="w-full px-6 pt-8 pb-4 flex justify-center items-center z-10 relative">
 <h1 className="text-xl font-black font-sans tracking-widest text-[#ef4444] flex items-center gap-3 uppercase animate-pulse">
 <AlertTriangle className="w-5 h-5 text-[#ef4444]" />
 SOS ACTIVE
 <AlertTriangle className="w-5 h-5 text-[#ef4444]" />
 </h1>
 </header>

 {/* Main beacon visualizer */}
 <main className="flex-1 flex flex-col items-center justify-center px-6 z-10 relative max-w-md mx-auto w-full">
 {/* Pulsing ring waves — 3 layers */}
 <div className="relative flex items-center justify-center w-64 h-64 mb-10">
 {/* Outermost slow pulse */}
 <div className="absolute inset-0 rounded-full bg-red-600 opacity-10 animate-ping" style={{ animationDuration: '1.8s' }} />
 {/* Mid ring */}
 <div className="absolute inset-6 rounded-full border border-red-600/30 animate-ping" style={{ animationDuration: '1.2s' }} />
 {/* Inner glow */}
 <div className="absolute inset-12 rounded-full bg-red-900/30 animate-pulse" />

 {/* Central SOS circle */}
 <div className="relative w-40 h-40 bg-[#c01e1e] rounded-full flex flex-col items-center justify-center
shadow-[0_0_80px_rgba(192,30,…
 <span className="text-white text-4xl font-black tracking-[0.15em] leading-none select-none">SOS</span>
 <span className="text-red-200 text-[10px] font-bold tracking-widest uppercase leading-none select-none">Emergency</span>
 </div>
 </div>

 {/* Temporary dynamic transmission feedback message */}
 {statusMessage && (
 <div className="w-full bg-stone-900 border border-red-500/30 rounded-2xl p-4 mb-5 text-center text-xs font-semibold
text-[#e6e8eb]">
 {statusMessage}
 </div>
 )}

 {/* Location Info Card */}
 <div className="w-full bg-[#2a0808]/90 rounded-2xl p-5 border border-red-900/30 mb-6 shadow-xl relative overflow-hidden
text-left">
 <div className="flex items-start gap-4">
 <MapPin className="w-6 h-6 text-red-500 shrink-0 mt-0.5" />
 <div className="space-y-3">
 <div>
 <h2 className="text-xs font-bold text-red-400 uppercase tracking-widest">
 Last Known Location
 </h2>
 <p className="text-base text-white font-bold leading-snug mt-1">
```

```
 {lastKnownLocation}
 </p>
 </div>
 <div className="flex items-center gap-2 bg-[#3f0f0f] border border-red-500/20 rounded-full px-3 py-1.5 w-fit">
 <Bluetooth className="w-3.5 h-3.5 text-red-400 fill-current" />
 <span className="text-[10px] font-bold tracking-wider text-red-300 uppercase">
 Shared via Bluetooth mesh
 </span>
 </div>
 </div>
 </div>
 </div>

 {/* Action buttons stack */}
 <div className="w-full flex flex-col gap-3">
 <button
 onClick={handleSendSms}
className="w-full h-14 bg-[#c82323] hover:bg-[#a81a1a] text-white font-bold rounded-2xl flex items-center justify-center
gap-3 shadow…
 <MessageSquare className="w-5 h-5 fill-current" />
 Send SMS Alert
 </button>
 <button
 onClick={handleBroadcastMesh}
className="w-full h-14 bg-[#60f8cb] hover:bg-[#4edab0] text-stone-950 font-bold rounded-2xl flex items-center justify-center
gap-3 sh…
 <Wifi className="w-5 h-5" />
 Broadcast via Mesh
 </button>
 </div>
 </main>

 {/* Footer actions */}
 <footer className="w-full px-6 pt-6 flex flex-col items-center gap-6 z-10 relative max-w-md mx-auto">
 <button
 onClick={onCancel}
className="font-sans font-bold text-sm text-zinc-400 hover:text-white underline underline-offset-4 p-2 transition-colors
uppercase trac…
 Cancel SOS
 </button>
 </footer>
 </div>
 );
}
```

QRScanner.tsx TypeScript/React | frontend/src/components/QRScanner.tsx

```
/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */

import React, { useState, useEffect, useRef } from 'react';
import { Camera, CameraOff, QrCode, Keyboard, AlertCircle, Info, RefreshCw } from 'lucide-react';
import { lookupQR } from '../services/api';

interface QRScannerProps {
 onScanSuccess: (data: any) => void;
 onCancel: () => void;
}

export default function QRScanner({ onScanSuccess, onCancel }: QRScannerProps) {
 const [stream, setStream] = useState<MediaStream | null>(null);
 const [cameraActive, setCameraActive] = useState(false);
 const [cameraError, setCameraError] = useState<string | null>(null);

 const [manualCode, setManualCode] = useState('');
 const [showManualInput, setShowManualInput] = useState(false);
 const [loading, setLoading] = useState(false);
 const [errorMsg, setErrorMsg] = useState<string | null>(null);

 const videoRef = useRef<HTMLVideoElement>(null);
 const localStreamRef = useRef<MediaStream | null>(null);

 const startCamera = async () => {
 try {
 setCameraError(null);

 // Stop any existing stream
 if (localStreamRef.current) {
 localStreamRef.current.getTracks().forEach(track => track.stop());
 }

```

```
  if (!navigator.mediaDevices || !navigator.mediaDevices.getUserMedia) {
  throw new Error('Camera access API is not supported in this browser environment or is blocked by security policies');
  }

  const mediaStream = await navigator.mediaDevices.getUserMedia({
  video: { facingMode: { ideal: 'environment' } }
  });

  localStreamRef.current = mediaStream;
  setStream(mediaStream);
  setCameraActive(true);

  setTimeout(() => {
  if (videoRef.current) {
  videoRef.current.srcObject = mediaStream;
  }
  }, 100);
  } catch (err: any) {
  let errorMessage = err.message || 'Camera access is blocked or unsupported';
  if (err.name === 'NotAllowedError' || err.name === 'PermissionDeniedError') {
  errorMessage = 'Camera permission is blocked. Please check your browser permissions and try again.';
  }
  setCameraError(errorMessage);
  setCameraActive(false);
  }
  };

  const stopCamera = () => {
  if (localStreamRef.current) {
  localStreamRef.current.getTracks().forEach(track => track.stop());
  localStreamRef.current = null;
  }
  setStream(null);
  setCameraActive(false);
  };

  // Request camera access on mount
  useEffect(() => {
  startCamera();
  return () => {
  stopCamera();
  };
  }, []);

  // Handle manual code or dropdown code lookup
  const handleVerifyCode = async (codeToVerify: string) => {
  if (!codeToVerify.trim()) return;
  setLoading(true);
  setErrorMsg(null);
  try {
  const result = await lookupQR(codeToVerify.trim().toUpperCase());
  onScanSuccess(result);
  } catch (err: any) {
  setErrorMsg(err.message || 'Verification failed. Please check the checkpoint code.');
  } finally {
  setLoading(false);
  }
  };

  return (
  <div className="relative w-full h-[calc(100vh-76px)] bg-zinc-950 flex flex-col justify-between overflow-hidden text-white
font-■■■">{/* LIVE VIDEO OR SIMULATED CORRIDOR VIEW ■■■ */}
  <div className="absolute inset-0 z-0">
  {cameraActive ? (
  <video
  ref={videoRef}
  autoPlay
  playsInline
  muted
  className="w-full h-full object-cover opacity-80"
  />
  ) : (
  <div className="relative w-full h-full bg-zinc-900/90 flex flex-col items-center justify-center p-6">
  {/* Camera unavailable fallback view */}
  <div className="relative z-10 text-center max-w-xs space-y-3 p-4 rounded-2xl bg-zinc-950/80 border border-zinc-850
backdrop-blur-lg">
  <Camera className="w-10 h-10 text-zinc-500 mx-auto animate-pulse" />
  <h3 className="text-sm font-bold text-zinc-300">Camera Unavailable</h3>
  <p className="text-[11px] text-zinc-500 font-semibold leading-normal">
  {cameraError ? `${cameraError}.` : 'Camera permission is required.'} Please enable camera access to continue.
  </p>

  <button
  type="button"
```

```
 onClick={startCamera}
className="w-full mt-2 py-2.5 bg-teal-600 hover:bg-teal-700 text-white font-bold text-xs uppercase tracking-wider rounded-xl tran…
 Enable Camera Scanner
 </button>
 </div>
 </div>
 )}
 </div>

 {/* ■■■ TOP BAR HUD INFO ■■■ */}
 <div className="relative z-10 w-full px-4 pt-6 flex justify-center">
 <div className="inline-flex items-center gap-2.5 bg-black/80 backdrop-blur-md rounded-full px-5 py-2.5 border border-zinc-800 text-zinc…
 <span className="text-[11px] font-bold tracking-wide">
 Align QR code or use the quick simulation dashboard below
 </span>
 </div>
 </div>

 {/* ■■■ VIEW FINDER BRACKET SIGHTS ■■■ */}
 <div className="relative z-10 flex-grow flex items-center justify-center p-8 select-none">
 <div className="relative w-64 h-64 flex items-center justify-center">
 {/* Viewfinder corner brackets */}
 <div className="absolute top-0 left-0 w-10 h-10 border-t-[4px] border-l-[4px] border-teal-400 rounded-tl-2xl"></div>
 <div className="absolute top-0 right-0 w-10 h-10 border-t-[4px] border-r-[4px] border-teal-400 rounded-tr-2xl"></div>
 <div className="absolute bottom-0 left-0 w-10 h-10 border-b-[4px] border-l-[4px] border-teal-400 rounded-bl-2xl"></div>
 <div className="absolute bottom-0 right-0 w-10 h-10 border-b-[4px] border-r-[4px] border-teal-400 rounded-br-2xl"></div>

 {/* Laser scanning line */}
 <div
 className="absolute top-0 left-0 w-full h-[3px] bg-teal-400 shadow-[0_0_12px_#2dd4bf] rounded-full"
 style={{
 animation: 'scan 2.5s ease-in-out infinite',
 animationName: 'qrScanMove'
 }}
 />

 <style>{`
 @keyframes qrScanMove {
 0%, 100% { top: 0%; opacity: 0.3; }
 50% { top: 100%; opacity: 1; }
 }
 `}</style>


 </div>
 </div>

 {/* ■■■ BOTTOM CONTROL INTERFACE ■■■ */}
 <div className="relative z-10 px-4 pb-28 max-w-md mx-auto w-full">
 {/* Error Dialog Banner */}
 {errorMsg && (
 <div className="bg-red-950/80 border border-red-500/30 backdrop-blur-md rounded-2xl p-3 mb-3 text-center text-xs font-bold text-red…
 <circle className="w-4 h-4 shrink-0" />
 <span>{errorMsg}</span>
 </div>
 )}

 <div className="bg-zinc-900/90 backdrop-blur-lg rounded-3xl p-5 shadow-2xl border border-zinc-800/50 space-y-4">

 {showManualInput ? (
 /* Manual Input form */
 <div className="space-y-3 text-center">
 <h4 className="text-xs font-extrabold text-zinc-400 uppercase tracking-widest">
 Manual Checkout Entry
 </h4>
 <div className="flex gap-2 justify-center">
 <input
 type="text"
 placeholder="e.g. QR-ENG-G01"
 value={manualCode}
 onChange={(e) => setManualCode(e.target.value)}
 className="bg-zinc-950 border border-zinc-800 text-center font-mono focus:border-teal-500 focus:ring-1 focus:ring-teal-500 outl…
 />
 </div>
 <div className="flex gap-2 pt-1">
 <button
 onClick={() => handleVerifyCode(manualCode)}
 disabled={loading}
 className="flex-1 bg-teal-600 hover:bg-teal-700 text-white font-bold py-3 rounded-xl text-xs uppercase tracking-wider transitio…
 Verify Code
```

```
 </button>
 <button
 onClick={() => {
 setShowManualInput(false);
 setErrorMsg(null);
 }}
 className="px-5 bg-zinc-800 hover:bg-zinc-750 text-zinc-300 font-bold py-3 rounded-xl text-xs uppercase tracking-wider
 transiti…
 Cancel
 </button>
 </div>
 </div>
 ) : (
 /* Simulator dropdown selector & toggles */
 <div className="space-y-4">
 <div className="space-y-2">
 <label className="text-[10px] font-black uppercase text-zinc-400 tracking-widest block text-left">
 Checkpoint Verification
 </label>
 <div className="grid grid-cols-1 gap-2">
 <input
 type="text"
 placeholder="Enter checkpoint code"
 value={manualCode}
 onChange={(e) => setManualCode(e.target.value.toUpperCase())}
 disabled={loading}
 className="w-full bg-zinc-950 border border-zinc-800 text-zinc-300 font-sans font-mono px-4 py-3 rounded-xl text-xs font-bold…
 />
 <button
 onClick={() => handleVerifyCode(manualCode)}
 disabled={loading || !manualCode.trim()}
 className="w-full bg-teal-600 hover:bg-teal-700 disabled:opacity-50 text-white font-bold py-3 rounded-xl text-xs uppercase tr…
 >
 {loading ? 'Verifying...' : 'Verify Checkpoint'}
 </button>
 </div>
 </div>

 <div className="flex justify-between items-center border-t border-zinc-850 pt-3.5">
 <button
 onClick={() => setShowManualInput(true)}
 className="text-xs font-extrabold text-teal-400 hover:text-teal-300 cursor-pointer flex items-center gap-1.5"
 >
 <Keyboard className="w-4 h-4" />
 Manual Entry
 </button>

 <button
 onClick={onCancel}
 className="text-xs font-bold text-zinc-400 hover:text-white cursor-pointer"
 >
 Cancel Scan
 </button>
 </div>
 </div>
 )}

 </div>
 </div>
 </div>
 );
}
```